ANIMATION & INTERACTIVITY (LECTURE)

REMINDERS

- HW00 was due Wednesday
  - You can still submit today and by using 2 late tokens
- Earn late tokens by handing in effortfully completed worksheets from class
- Check-in due before next class
- HW01 has been released, due Weds
  - START EARLY!
- *Sunday Review Session each Sunday 3-5pm in AGH 214*
- Recitation continues next week

LEARNING OBJECTIVES

- Get introduced to a basic form of iteration with the while loop
- Write PennDraw programs that create moving images
- Write programs that react in real time to user inputs
  - Understand how to check for & use mouse input with PennDraw
  - Understand how to check for & use keyboard input with PennDraw

WHILE LOOPS

Last time we talked about `if` statements, that only run the code they contain when some **boolean expression** evaluates to True

But what if we want something to continue *as long as* a boolean expression evaluates to True?

```
while it_is_raining:
    I will stay inside
    I will play video games
I will go outside
```

THE WHILE TRUE LOOP

`while`, like `if`, is a keyword that allows us to control the flow of a program.

```
while expression:
  do_this()
  do_that()
```

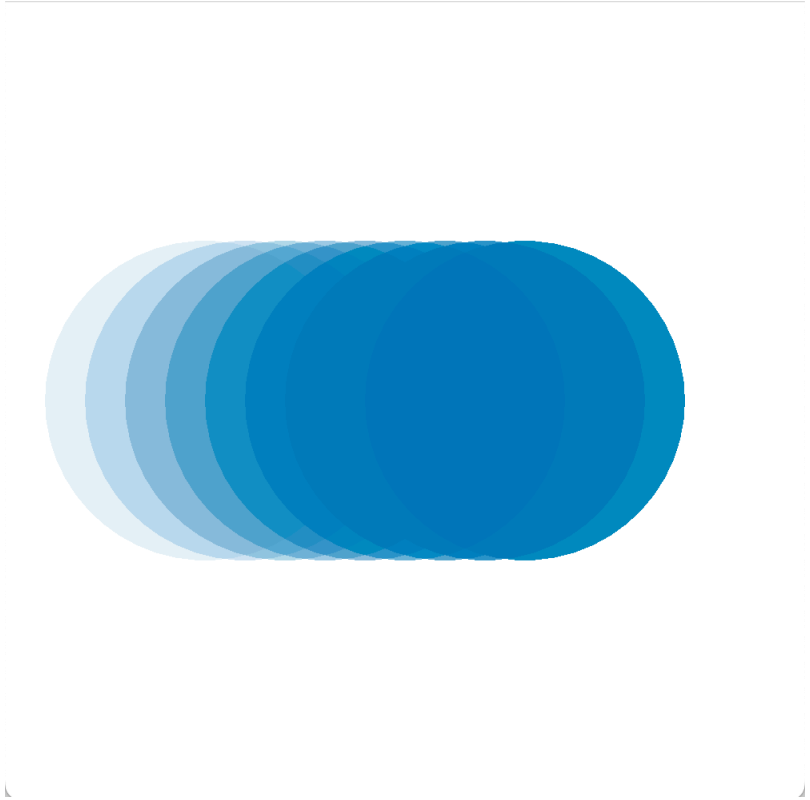
When we reach the `while`, we test its condition. If `True`, we execute the statements in the body. Then, we **test the condition again**.

- Different from a conditional (`if`), where we only test ONCE!
- If the `expression` is literally `True`, we will loop here forever...

ANIMATION IN PENNDRAW

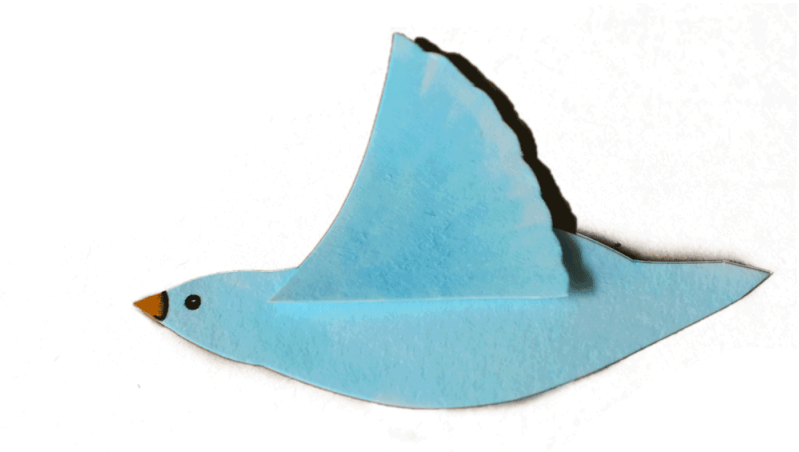
ANIMATION AS A MODEL

- For static images, we picked the positions, sizes, & colors of our shapes **once**.
- Smooth animation is achieved by showing a lot of similar images very quickly
  - A "flipbook" model
  - Shapes will change position, size, color slightly with each update



ANIMATION AS A MODEL

- For static images, we picked the positions, sizes, & colors of our shapes **once**.
- Smooth animation is achieved by showing a lot of similar images very quickly
  - A "flipbook" model
  - Shapes will change position, size, color slightly with each update



FRAMES

- Animations as "flipbooks" require that we draw many images per second
  - Call these images **frames**
  - More frames per second *("higher FPS")* ➡️ smoother animation
- A frame consists of a set of shapes rendered at a specific moment in time
  - Different frames typically have the same shapes but drawn in different ways

DRAWING FRAMES IN PENNDRAW

Animations usually include a *setup* and an *animation loop.*

```
import penndraw as pd
# SETUP: This code is run just one time!
pd.set_canvas_size(500, 500)
x_center = 0.5
y_center = 0.5
half_side = 0.1
pd.set_pen_color(pd.HSS_BLUE)
# ANIMATION LOOP: This code is run many times per second, over and over and over...
while True:
    pd.clear()
    pd.filled_square(x_center, y_center, half_side)
    x_center += 0.01
    if x_center > 1 + half_side:
        x_center = -half_side
    pd.advance() # Necessary at the end of the loop
```

ANIMATING: THE SETUP

Animated programs usually start with a "setup" block, where we:

- choose settings, like canvas size
- declare variables that we will use to draw each frame of the animation
    - variables may vary, but we can pick initial values (how the animation starts)
- do anything that needs to happen **only one time**

```
import penndraw as pd
pd.set_canvas_size(500, 500)
x_center = 0.5
y_center = 0.5
half_side = 0.1
pd.set_pen_color(pd.HSS_BLUE)
```

ANIMATING: THE `WHILE TRUE` LOOP

The body of the `while` loop is our *animation loop*:

- runs many times per second
- runs indefinitely until the program is manually stopped
- allows us to draw many frames per second, changing something slightly each time

```
# ANIMATION LOOP: This code is run many times per second, over and over and over...
while True:
    pd.clear()
    pd.filled_square(x_center, y_center, half_side)
    x_center += 0.01
    if x_center > 1 + half_side:
        x_center = -half_side
    pd.advance() # Necessary at the end of the loop
```

ANIMATING: ANIMATION LOOP RECIPE

For each frame,

1. decide whether to clear the screen
    1. Clearing the screen ➡ all previous shapes disappear, and only the next frame will be visible
    2. Not clearing ➡ the next frame will be drawn on top of the current frame, other frames might still be visible

    …

ANIMATING: ANIMATION LOOP RECIPE

2. draw the next frame based on current properties of the shapes
    1. "properties of the shapes" usually stored in variables
3. update the properties of the shapes for the next frame
4. `pd.advance()`  make everything show up on screen
    1. Always need this at the end of the loop.

PRACTICE: ANIMATION LOOP

Describing in words, what will the output of the following code be?

L11:

```
import penndraw as pd
x_center = 0.5

while True:
  pd.clear()
  pd.filled_square(x_center, 0.5, 0.1)
  pd.advance()

print("tada!")
```

PRACTICE: ANIMATION LOOP

How could we change this code so that the square appears slightly to the right of the center? Slightly to the right of that?

```
import penndraw as pd
x_center = 0.5

while True:
  pd.clear()
  pd.filled_square(x_center, 0.5, 0.1)
  pd.advance()

print("tada!")
```

EXAMPLE ANIMATION LOOP: SLIDING SQUARE

Produces a square that slides left-to-right across the canvas.

```
import penndraw as pd
x_center = 0.5 # SETUP
while True:
    pd.clear()                                # 1. clear the screen
    pd.filled_square(x_center, 0.5, 0.1)      # 2. draw this frame
    x_center += 0.01                          # 3. update shapes for next frame
    pd.advance()                              # 4. pd.advance()
```

EXAMPLE ANIMATION LOOP: BOUNCING SQUARE

What boolean expression can we use to check if the right side of the square has passed the edge of the screen? (Write the boolean expression)

C12 (for extra scratch space)

```
import penndraw as pd
x_center = 0.5 # SETUP
while True:
    pd.clear()                                  # 1. clear the screen
    pd.filled_square(x_center, 0.5, 0.1)        # 2. draw this frame
    x_center += 0.01                            # 3. update shapes for next frame
    pd.advance()                                # 4. pd.advance()
```

EXAMPLE ANIMATION LOOP: BOUNCING SQUARE

```
import penndraw as pd
x_center = 0.5 # SETUP
while True:
    pd.clear()                                  # 1. clear the screen
    pd.filled_square(x_center, 0.5, 0.1)  # 2. draw this frame

    if x_center > 0.9:                          # 3. update shapes for next frame
        x_center -= 0.01
    else:
        x_center += 0.01
    pd.advance()                                # 4. pd.advance()
```

HOW DO WE FIX THIS?

```
import penndraw as pd
x_center = 0.5 # SETUP
while True:
    pd.clear()                                  # 1. clear the screen
    pd.filled_square(x_center, 0.5, 0.1)  # 2. draw this frame

    if x_center > 0.9:                          # 3. update shapes for next frame
        x_center -= 0.01
    else:
        x_center += 0.01
    pd.advance()                                # 4. pd.advance()
```

MOUSE INPUT

CLICKING INTO PLACE

PennDraw includes a few tools useful for handling cursor position & clicking:

FUNCTION	RETURN TYPE	DESCRIPTION
`pd.mouse_pressed()`	`bool`	Returns `True` if the mouse is being held this frame.
`pd.mouse_x()`	`float`	Returns the **x coordinate** of the mouse's current location, e.g. `0.9` or `0.1443`
`pd.mouse_y()`	`float`	Returns the **y coordinate** of the mouse's current location, e.g. `0.9` or `0.1443`

CLICK COUNTER

```
import penndraw as pd
num_clicks = 0;

while True:
  pd.clear()
  pd.text(0.5, 0.5, f"Number of Clicks: {num_clicks}")
  if pd.mouse_pressed():
    num_clicks = num_clicks + 1
  pd.advance()
```

Each frame, if we click the mouse, increment `num_clicks`.

BUT WHAT TO DO WHEN THINGS GO WRONG?

CLICK COUNTER

```
import penndraw as pd

while True:
  num_clicks = 0;
  pd.clear()
  pd.text(0.5, 0.5, f"Number of Clicks: {num_clicks}")
  if pd.mouse_pressed():
    num_clicks = num_clicks + 1
  pd.advance()
```

Why doesn't this work? :(

FOLLOWING SQUARE

```
import penndraw as pd

pd.set_canvas_size(500, 500)
x_center = 0.5
y_center = 0.5
half_side = 0.1
pd.set_pen_color(pd.HSS_BLUE)

while True:
  pd.clear()
  x_center = pd.mouse_x()   # Ask for the x-coordinate of the cursor
  y_center = pd.mouse_y()   # Ask for the y-coordinate of the cursor
  if pd.mouse_pressed():    # Ask whether the mouse is being clicked
    pd.set_pen_color(pd.HSS_RED)
  else:
    pd.set_pen_color(pd.HSS_BLUE)
  pd.filled_square(x_center, y_center, half_side)
  pd.advance()
```

CALCULATING DISTANCE

Since your mouse has a specific location on the screen, it might become relevant to find the distance between two points.

The formula for the distance between (x_1, y_1) and (x_2, y_2) is the following:

$$\text{distance} = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

The `math` library in Python contains a `sqrt()` function that calculates the square root of its input.

ACTIVITY: CALCULATING DISTANCE

C14

Given variables `x_one, y_one, x_two, y_two`, can you write a snippet of Python that calculates the distance between points `(x_one, y_one)` and `(x_two, y_two)`?

KEYBOARD INPUT

"KEYS" TO SUCCESS

User key presses can also be registered!

FUNCTION	RETURN TYPE	DESCRIPTION
`pd.has_next_key_typed()`	`bool`	Returns `True` if a key is currently being held
`pd.next_key_typed()`	`str`	Returns the key currently being held down.

🚨 Don't use `next_key_typed()` without checking `has_next_key_typed()` first! 🚨

CHECKING KEYS

The value produced by `pd.next_key_typed()` is a string with a length of one

```
key = pd.next_key_typed()
```

- To see if a specific key was pressed:
  - `if key == "x": ...` or `if key == "!": ...`
- To see if the key was a lowercase letter:
  - `if "a" <= key <= "z": ...`
- To see if the key was a digit:
  - `if "0" <= key <= "9": ...`

LIGHT SWITCH

Why does this program crash? L13

```
import penndraw as pd
on = False
while True:
  if not on:
    pd.clear(pd.BLACK)
  else:
    pd.clear(pd.YELLOW)

  key = pd.next_key_typed()
  if key == "x":
    on = not on
  pd.advance()
```

LIGHT SWITCH

Why does this program crash?

```
import penndraw as pd
on = False
while True:
  if not on:
    pd.clear(pd.BLACK)
  else:
    pd.clear(pd.YELLOW)

  if pd.has_next_key_typed():
    key = pd.next_key_typed()
    if key == "x":
      on = not on
  pd.advance()
```

RANDOMNESS

RANDOMNESS

What if our program didn't always draw the same picture each time?

```
import random
print("Picking a random number between 0 and 0.99999...")
my_float = random.random()
print("Picking a random integer between 1 and 100.")
my_int = random.randint(1, 100)
print("my_float:", my_float, "my_int:", my_int)
```

🖨️👇 (for example)

```
Picking a random number between 0 and 0.99999...
Picking a random integer between 1 and 100.
my_float: 0.3025819686483993 7 my_int: 13
```

PICKING A RANDOM COLOR

How can we fill in the blank with lines of code so that we pick a random color for our square each time?

```
import random
import penndraw as pd

#  PUT SOME CODE HERE!

pd.set_pen_color(red, green, blue)
pd.filled_circle(0.5, 0.5, 0.2)
pd.run()
```

PICKING A RANDOM COLOR

How can we fill in the blank with lines of code so that we pick a random color for our square each time?

```
import random
import penndraw as pd

red = random.randint(0, 255)
green = random.randint(0, 255)
blue = random.randint(0, 255)

pd.set_pen_color(red, green, blue)
pd.filled_circle(0.5, 0.5, 0.2)
pd.run()
```

RANDOM EVENTS

- Each call to `random.random()` gives a result between `0` and `1` where each is equally likely.
- Therefore, there's a 100% chance the number generated is less than `1`
- There's a 90% chance the number generated is less than `0.9`
- There's an 80% chance the number generated is less than `0.8`
- There's an 53.4% chance the number generated is less than `0.534`

➡️ to simulate an event that happens $x\%$ of the time, draw a random number and check if it falls in the range of $(0, \frac{x}{100}]$